

Hospital Management System

Full QA Testing Report

Django 6.1 hospital management system: authentication, role-based access, all four portals (patient / doctor / receptionist / admin), billing, form validation, security, responsive UI, and performance — tested live against the running application and fixed end to end.

Target	D:\HMS\Hospital-Management-System
Stack	Django 6.1 / MySQL 8.4
Date	2026-09-04
Method	Live browser + HTTP testing, 4 parallel code-review passes, all fixes re-verified against the running app

ALL FINDINGS RESOLVED

Every issue below — all 3 Critical, 6 High, and 11 Medium findings, plus every fixable Low — has been fixed and individually re-verified against the running app. Each finding that required a code change is marked **FIXED**. Three items were deliberately left as notes (see closing summary).

3	6	11	10 (7 fixed)	13
Critical	High	Medium	Low	Verified Pass

Methodology: every finding was either live-reproduced against the running dev server (browser automation + direct HTTP/test-client requests) or confirmed by reading the current source. Every fix was applied to the actual source, then re-verified with a fresh request (and, for the double-booking race, 5 genuinely concurrent threads) — not just re-read. All test data created during testing and fixing (orphaned accounts, temporary bookings, availability edits, deactivated accounts) was rolled back.

1. Authentication

Login, logout, registration, password reset, and the two-step email-OTP flow were tested end to end, including deliberately breaking each one.

Blank Date of Birth on registration crashed the site and permanently killed the email address

CRITICAL

FIXED

STEPS	POST to /register/ with date_of_birth omitted (bypasses only the HTML form's required attribute).
EXPECTED	A validation error; no account created.
ACTUAL (BEFORE FIX)	Full Django debug 500 traceback page; the User row was already committed before the crash, permanently orphaning that email (no delete-account feature exists).
FIX APPLIED	patient.views.register now normalizes date_of_birth to None when blank, and wraps User+Patient creation in transaction.atomic() so any downstream failure rolls back cleanly. Live-reproduced the crash first, then confirmed the fix, then deleted the one orphaned test account.

All 4 "change password" forms crashed on a blank new password

HIGH

FIXED

STEPS	Authenticated, POST the correct current_password but omit new_password/confirm_new_password.
EXPECTED	"Please fill in all fields" style error.
ACTUAL (BEFORE FIX)	Only an equality check existed (new == confirm), which two Nones pass; validate_password(None, user) then threw an uncaught TypeError.
FIX APPLIED	Added a blank-field guard to patient/doctor/receptionist/adminpanel change_password, mirroring reset_password_confirm's existing pattern. Verified: guard triggers with no crash; passwords unchanged.

Duplicate-email race on registration/onboarding had no DB-level fallback

MEDIUM

FIXED

STEPS	Two near-simultaneous signups with the same email (self-register, walk-in registration, or admin onboarding).
EXPECTED	Second one rejected cleanly.
ACTUAL (BEFORE FIX)	Both checked .exists() before creating; the loser hit the DB unique constraint directly, an uncaught IntegrityError.
FIX APPLIED	Wrapped all 4 create_user() call sites in transaction.atomic() + except IntegrityError, showing "Email already registered." instead of crashing.

Login, OTP lifecycle, and lockout all work correctly end to end

PASS

VERIFIED PASS

VERIFIED

Wrong password / non-existent email both return the same generic error (no account enumeration). With OTP on: attempts-remaining counter decrements correctly, 3rd wrong code sets a real 10-minute lockout and shows the correct message (confirmed via a clean curl session). Correct code logs in and redirects to the right role dashboard. Logout calls real `django.contrib.auth.logout()`.

Logged-in users could still open `/login/` and `/register/`

LOW

FIXED

ACTUAL (BEFORE FIX)

No redirect-if-authenticated guard. No security impact, just a UX inconsistency.

FIX APPLIED

`login()` and `register()` now redirect an already-authenticated visitor straight to their role dashboard. Verified for both a patient and an admin session; anonymous visitors still see the real form.

2. Role-Based Access

Every view across all 5 apps was checked for its decorator, then cross-checked live by logging in as each role and hitting other roles' URLs directly.

No `patient_required` decorator — patient views relied on convention, not enforcement

MEDIUM

FIXED

STEPS

Log in as a doctor, receptionist, or admin; visit `/patient-profile/` or `/book-appointment/`.

EXPECTED

Redirected away — patient-only area.

ACTUAL (BEFORE FIX)

Fully accessible; `Patient.objects.get_or_create(user=request.user)` silently created a Patient record for a doctor/receptionist/admin account. Live evidence: the seeded admin account "Vikrant Mehta" had real appointments and bills against it.

FIX APPLIED

Added a `patient_required` decorator (mirroring `doctor_required/receptionist_required/admin_required`) and applied it to all 14 patient-app views. Live-confirmed an admin hitting these URLs is now redirected to home. The stale admin-account Patient/Appointment/Billing rows were deleted for real.

Doctor / Receptionist / Admin decorator coverage is complete and correct

PASS

VERIFIED PASS

VERIFIED

15/15 doctor views, 21/21 receptionist views, 22/22 admin views carry a role decorator checking `request.user.role`. Live-tested: the account-deactivation privilege-escalation guard correctly refused to deactivate the admin account itself; toggling a doctor's active status worked and round-tripped cleanly.

3. Patient Module

Profile, appointment booking, medical records/prescriptions, and billing — tested as seeded patient Anita Sharma.

Patients could book — and pay for — an appointment dated in the past

CRITICAL

FIXED

STEPS

Submit `appointment_date=2020-01-15` for any doctor/slot (bypasses only a client-side JS min).

EXPECTED

Rejected — dates before today aren't bookable.

**ACTUAL
(BEFORE
FIX)**

Booking sailed through to the real confirm-and-pay screen showing "Date: Jan 15, 2020" and the fee due. `Doctor.get_available_slots()` only special-cased today; it never rejected a wholly past date.

**FIX
APPLIED**

Added a `date.today()` guard as the very first check inside `Doctor.get_available_slots()` — the single source both booking views already call. Live-reproduced to the payment screen before the fix; confirmed rejected after, for both the patient and receptionist flows.

Profile name field accepted raw script tags — safely escaped everywhere shown

LOW

FIXED

STEPS

Set First Name to `alert('xss')` and save.

ACTUAL

Saved verbatim, but Django's template auto-escaping neutralized it everywhere (confirmed via DOM inspection). Not a security bug, but no format validation existed.

**FIX
APPLIED**

Added `is_valid_name()` (letters/spaces/hyphens/apostrophes) at all 4 account-creation views. Verified: markup/digit "names" rejected, real names with hyphens/apostrophes still accepted.

Cross-patient data access (IDOR) is correctly blocked everywhere checked

PASS

VERIFIED PASS

VERIFIED

Requesting another patient's billing receipt returns a clean 404, not the data. Same ownership-scoped pattern confirmed on `cancel_appointment`, `dismiss_notification`, `my_appointments`, `my_billing`.

Patient-facing pages silently dropped error messages

MEDIUM

FIXED

STEPS	Submit the Contact form with every field blank.
EXPECTED	"Please fill in all required fields" shown on the page.
ACTUAL (BEFORE FIX)	patient/base.html had no {% if messages %} block at all; the message was genuinely queued server-side but never shown.
FIX APPLIED	Added a shared {% block messages %} to patient/base.html with correct Bootstrap tag-mapping, and removed the now-redundant duplicate inline blocks from 8 patient templates. Live-confirmed the contact error now shows, correctly styled.

4. Doctor Module

Dashboard, availability, prescriptions, patient records — tested as Dr. Sarah Mitchell.

Availability form accepted an inverted time range with zero feedback

HIGH

FIXED

STEPS	Set Monday's hours to Start 17:00, End 09:00, Save.
EXPECTED	Rejected, or auto-corrected with a warning.
ACTUAL (BEFORE FIX)	Saved silently; get_available_slots() then produced zero bookable slots that day with no explanation.
FIX APPLIED	Validates every day's start < end before writing any row, rejecting an inverted range with a clear error. Live-reproduced the silent save, confirmed rejected after, confirmed a valid full-week submission still saves.

Negative consultation fee crashed the profile save with a raw DB error

MEDIUM

FIXED

STEPS	Edit profile, set Consultation Fee to -500, submit.
EXPECTED	"Fee must be positive" validation error.
ACTUAL (BEFORE FIX)	Uncaught django.db.utils.DataError (unsigned MySQL column), only a bypassable client-side min="0".
FIX APPLIED	Validates consultation_fee/experience_years ≥ 0 before save on both doctor.edit_profile and admin's add_doctor. Live-reproduced the crash via test client, confirmed rejected after with fee left unchanged.

New prescription's linked appointment wasn't ownership-checked

LOW

FIXED

ACTUAL
(BEFORE
FIX)

The patient was re-validated against the doctor's own patients, but the optional appointment_id was never checked to belong to that same doctor/patient pair.

FIX
APPLIED

add_prescription now verifies the appointment_id belongs to the same doctor+patient before creating. Live-tested a tampered cross-doctor appointment_id and confirmed rejection; valid submissions still work.

Dashboard, patient records, and role gating all correct

PASS

VERIFIED PASS

VERIFIED

Dashboard stats rendered correctly; every doctor view is scoped to the requesting doctor's own record.

5. Receptionist Module

Patient registration and appointment management — tested as Priya Nair.

Front-desk booking date picker had no past-date restriction at all

HIGH

FIXED

STEPS

Open the receptionist "Book Appointment" page, pick a date on the native calendar.

EXPECTED

Calendar refuses to offer dates before today.

ACTUAL
(BEFORE
FIX)

The date input's min attribute was empty; a receptionist needed zero effort to pick a backdated appointment, which then succeeded server-side.

FIX
APPLIED

Same server-side fix as the Patient Module past-date bug covers this endpoint too; added a min="{% now 'Y-m-d' %}" attribute to the date input as defense-in-depth.

Doctor directory computed each row's appointment count with a separate query (N+1)

MEDIUM

FIXED

ACTUAL
(BEFORE
FIX)

One Appointment.objects.filter(...).count() per doctor in a loop.

FIX
APPLIED

Replaced with a single grouped .values('doctor').annotate(Count('id')) query. Verified: query count for the page is now flat (10 total) instead of growing per doctor.

Receptionist booking's patient dropdown rendered every patient in the system, unbounded

HIGH

FIXED

**ACTUAL
(BEFORE
FIX)**

Every patient rendered as a plain <option> with no search or cap — usable today, would become unusable well before four figures.

**FIX
APPLIED**

Replaced with a live search box backed by a new receptionist.views.search_patients JSON endpoint (capped at 20 results). Live-tested in the actual browser: typing 'anita' shows a dropdown match, clicking it fills the hidden field and the booking summary; a full booking still completes end to end.

Registration, dashboard, and role gating correct

PASS

VERIFIED PASS

VERIFIED

Dashboard stats correct; receptionist_required correctly blocks other roles.

6. Admin Module

User management, OTP settings, reports — tested as Vikrant Mehta.

Account activation/deactivation and OTP toggle both verified live, in both directions

PASS

VERIFIED PASS

VERIFIED

Deactivated and reactivated a real doctor account via the toggle endpoint — took effect immediately. Attempting to deactivate the admin's own account was correctly refused. The Security Settings OTP switch flips the DB-backed setting and takes effect on the next login attempt.

Dashboard revenue figure summed in Python instead of the database

MEDIUM

FIXED

**ACTUAL
(BEFORE
FIX)**

sum(a.doctor.consultation_fee for a in completed_appointments) loaded every completed appointment into memory.

**FIX
APPLIED**

Replaced with .aggregate(Sum('doctor__consultation_fee')). Verified: identical ■1200 result before and after.

Stale test data from earlier work was still in the seeded database

LOW

FIXED

**ACTUAL
(BEFORE
FIX)**

The admin account "Vikrant Mehta" carried a real pending bill and appointments — leftover from earlier testing, and the same data that made the RBAC gap concretely visible.

**FIX
APPLIED**

Deleted the stale Patient/Appointment/Billing rows tied to the admin account (cascaded cleanly); the admin account itself was left untouched.

7. Billing & Payments

Bill generation, the demo card-payment flow, no-show fees, and receipts.

Backdated bookings meant backdated, fully-paid bills

HIGH

FIXED

**ACTUAL
(BEFORE
FIX)**

Direct consequence of the past-date booking bug: since the demo payment flow treats "submitted" as "paid," a backdated Appointment + fully-paid Billing row for a visit that never happened was one click away.

**FIX
APPLIED**

Resolved by the same Doctor.get_available_slots() fix under Patient Module.

Double-booking's DB-level safety net was removed and never replaced

MEDIUM

FIXED

**ACTUAL
(BEFORE
FIX)**

A prior fix correctly dropped Appointment's unique_together, leaving the try/except IntegrityError fallback as the only protection — and two near-simultaneous bookings could both pass the availability check and both succeed.

**FIX
APPLIED**

Wrapped the availability-recheck-then-create sequence in transaction.atomic() with Doctor.objects.select_for_update() in both booking views. Proved this with a real concurrency test: 5 threads hit the identical doctor/date/slot simultaneously — exactly 1 succeeded, 4 got "slot already taken," and only 1 Appointment row actually existed afterward.

Card-payment validation, no-show billing, and receipt access control all check out

PASS

VERIFIED PASS

VERIFIED

The demo card form correctly blocks submission with blank fields; billing receipts are ownership-scoped per role.

8. Form Validation

This project has no forms.py by design — every field is read manually via request.POST.get(), so form-layer protections Django would normally provide never run.

Doctor/receptionist/patient/admin profile edits lacked the required-field guard their onboarding forms already have

HIGH

FIXED

ACTUAL (BEFORE FIX)

edit_profile views didn't guard department/email/name fields before saving; omitting them in a raw POST raised an uncaught IntegrityError.

FIX APPLIED

Added required-field guards to doctor/receptionist/patient/admin profile-edit views, mirroring add_doctor's existing pattern. Live-tested missing-field rejection on all 5 sites with no crash and no unintended changes.

Contact form errors generated but never shown

MEDIUM

FIXED

NOTE

Same root cause and fix as the Patient Module finding above.

Error messages rendered with no red styling on 3 of the 4 portals

MEDIUM

FIXED

ACTUAL (BEFORE FIX)

The naive alert alert-{{ message.tags }} pattern produced the invalid Bootstrap class alert-error for Django's 'error' tag on adminpanel/base.html, doctor/base.html, receptionist/base.html, and 4 patient templates.

FIX APPLIED

Applied the correct {% if message.tags == 'error' %}alert-danger{% endif %} mapping (already used correctly elsewhere) to all remaining base templates. Live-verified all 4 portals now render error messages in red.

Duplicate email and weak-password rejection both work correctly

PASS

VERIFIED PASS

VERIFIED

Re-registering an existing email is rejected with no account created. Submitting "password" is rejected as too common, matching all 9 password-validation call sites.

9. Security

SQL injection, XSS, CSRF, IDOR, and unauthorized access — the highest-priority section of this audit.

Live, unauthenticated stored-XSS files were sitting on disk from a previous test session

CRITICAL

FIXED

STEPS	As any anonymous visitor, request /media/patients/profile_images/poc_upload_test.html (and 4 sibling files).
EXPECTED	404, or served as an inert download.
ACTUAL (BEFORE FIX)	Django served the file as text/html; the embedded <script> payload executed at the application's own origin — a working, unauthenticated stored-XSS exploit, independent of the upload-time validation fix.
FIX APPLIED	Deleted all 5 files immediately. Confirmed the URL now returns 404.

Open-redirect defense-in-depth gap on several "next" params

LOW

FIXED

ACTUAL (BEFORE FIX)	redirect(request.POST.get('next') or fallback) in 9 call sites didn't validate next was a safe local URL.
FIX APPLIED	Added a shared safe_next_redirect() helper (url_has_allowed_host_and_scheme) and applied it to all 9 sites. Live-tested: a malicious next=https://evil.example correctly falls back to the safe default, while a genuine safe relative next is still honored.

Self-XSS-only escaping mistake in two doctor-portal JS files

LOW

FIXED

ACTUAL (BEFORE FIX)	The prescription-history medicine modal and the availability blocked-date list both built HTML via innerHTML string concatenation — self-XSS only today, but a real escaping mistake.
FIX APPLIED	Rewrote both to build the DOM via createElement/textContent instead. Live-tested with an actual payload in both spots and confirmed it now renders as escaped text, not executable markup.

DEBUG=True by default; verbose error pages confirmed reachable

LOW

NOTED, NOT CHANGED

ACTUAL	Correct and expected for this local-dev environment — every crash found during testing returned a full technical traceback. Flagged so it's explicitly checked before any real deployment, not because it's wrong for today's environment.
--------	--

SQL injection, CSRF, reflected XSS, and cross-user IDOR: all clean under direct testing

PASS

VERIFIED PASS

VERIFIED

No raw SQL anywhere in the codebase. A token-less POST to /login/ was rejected with a clean 403. A literal <script> saved into the profile name field rendered as safely escaped text everywhere. Every cross-user object-access attempt made during this audit was correctly blocked.

10. UI/UX & Responsive

Desktop, tablet, and mobile viewports — checked on the public homepage and the data-heaviest admin pages.

No horizontal overflow or broken layout found at 375px mobile width

PASS

VERIFIED PASS

VERIFIED

Homepage hero, admin dashboard stat cards, and the admin patient list all reflowed cleanly into single-column mobile layouts with zero horizontal scroll.

Unstyled error alerts were also an accessibility/UX issue, not just cosmetic

MEDIUM

FIXED

NOTE

Same root cause and fix as the Form Validation finding above.

OTP page briefly flashes "--:--" before the real countdown starts

LOW

NOTED, NOT CHANGED

ACTUAL

Cosmetic only — self-corrects to the real countdown within about a second as client JS initializes.

11. Performance

Page load, query counts, and behavior under the current dataset (13 patients, 10 doctors, 24 accounts), measured via captured query counts, not estimated.

At current data volume, every page measured is fast

PASS

VERIFIED PASS

VERIFIED

Admin dashboard: ~180ms / 23 queries. Patient/appointment/billing lists: 18–28ms / 7–8 queries each. No user-visible slowness anywhere tested.

Zero database indexes existed anywhere in the project

HIGH

FIXED

**ACTUAL
(BEFORE
FIX)**

No db_index=True or Meta.indexes anywhere. Most consequential: Appointment.appointment_date, filtered/ordered on in nearly every dashboard and "today" view.

**FIX
APPLIED**

Added db_index=True to Appointment.appointment_date, Appointment.status, Billing.status, and Patient.created_at (migration 0014). Confirmed all 4 indexes exist live in MySQL via SHOW INDEX.

Front-desk booking's patient dropdown listed every patient, unpaginated

HIGH

FIXED

NOTE

Same fix as the Receptionist Module finding above.

Several N+1 query patterns confirmed by code

MEDIUM

FIXED

**ACTUAL
(BEFORE
FIX)**

Receptionist's doctor directory, the prescription form's patient-name lookup, and admin's patient list each ran one extra query per row instead of a joined queryset.

**FIX
APPLIED**

Added the missing select_related()/replaced the per-row loop with a grouped annotate() in all 3 spots. Verified flat, non-growing query counts on each page.

Revenue and outstanding-bill totals summed in Python, not the database

MEDIUM

FIXED

NOTE

Same fix as the Admin Module finding above.

Previously-audited pagination and server-side filtering hold up with no regression

PASS

VERIFIED PASS

VERIFIED

All 10 previously-paginated list views still page correctly with deterministic ordering.

Overall Testing Summary

Verdict: PASS.

Every finding from the original audit that required a code change has been fixed and individually re-verified against the running app — not just re-read. The three release-blockers are gone: the 5 leftover proof-of-concept exploit files are deleted and confirmed 404 (was live XSS); registering with a blank date of birth now shows a

friendly error with no account created (was a 500 crash + permanently orphaned email); and both booking flows now reject a past appointment date at the single shared source (`Doctor.get_available_slots()`), live-tested end to end on both the patient and receptionist flows.

Everything else followed the same pattern — fix, then prove it with a real request against the live app: the double-booking race is now closed by a `select_for_update()` lock (proven with 5 genuinely concurrent threads — exactly 1 succeeded), a `patient_required` decorator now exists and blocks other roles from patient-only actions, 4 new database indexes are live in MySQL, the receptionist's patient dropdown is now a real search endpoint instead of rendering every patient, every one of the 4 portals' error messages now shows with correct red styling, and the two innerHTML/XSS-fragile spots in the doctor portal's JS were rewritten to build the DOM safely instead of concatenating strings.

Three items were deliberately left as notes rather than fixes: `DEBUG=True` is correct for this local dev environment (just confirm it's overridden before any real deployment); the Django admin site's separate `is_staff`-based privilege plane is an architectural observation, not an active vulnerability (the app never grants `is_staff`); and the brief "--:--" flash on the OTP countdown before its JS initializes is cosmetic. All test data created while fixing and re-verifying was rolled back, and the stale admin-account patient/billing data flagged in the original audit has now been deleted for real.

Audited and fixed 2026-09-04 · 4 static code-review passes + live browser/HTTP testing against the running dev server · all fixes individually re-verified · all test data rolled back